

Why “No Bug” Was Still a Win

By [erynom](#) • on [Bugcrowd](#)

An honest walk through testing a password reset flow. The setup, the checks, and the lesson that stuck with me.

Web security • Bug bounty • Learning in public

Not every session ends with a finding, and this is one of those. I picked a real program on Bugcrowd, set everything up, and spent a sitting poking at one feature. This is a plain account of what I did, what tripped me up, and the one takeaway that stuck with me the most.

Spoiler: I didn't find a vulnerability. And by the end of the day I was genuinely fine with that. Here's why.

Getting set up

Before you can test anything, you need an account on the platform and a working lab on your own machine. I signed up on Bugcrowd as a researcher, generated my email alias, and created a test account on the target using that alias. On the machine side I used Kali Linux with Burp Suite, the proxy that lets you see and replay every request your browser makes.

The real hurdle wasn't hacking anything. It was getting the proxy to trust the site's HTTPS. Until I installed Burp's certificate, the login page kept throwing errors. Small thing, but it taught me a habit: get your tooling clean first, then test. Logging in through a proxy that isn't set up properly just creates noise you'll waste time chasing.

The part nobody warns you about: scope

Once Burp was capturing traffic, I clicked around the app and watched the requests pile up. And here's what surprised me: a modern web app talks to a lot of different domains. Payment providers, cookie consent services, internal subdomains, static file hosts, all firing off in the background.

Most of those are **out of scope**. The program tells you exactly which targets you're allowed to test, and everything else is off limits, including services owned by other companies that aren't even the target's to authorize. Learning to read that traffic and separate “I can test this” from “hands off” turned out to be a real skill on its own, and one a lot of beginners skip straight past.

Half of doing this properly is knowing what you're *not* allowed to touch.

Picking something I could actually test

The most valuable features needed a physical device I didn't own, so a lot of the app was locked to me. Rather than bang my head against locked doors, I picked something I could fully test with just my own account: the **password reset flow**. It's a good target for a beginner because the logic is easy to reason about, and you can test it from start to finish without ever touching another user.

I focused on three simple questions:

- Is the reset token random and strong, or could someone guess it?
- Does the token leak any personal data if you decode it?
- Can the same reset link be used more than once?

Looking inside the token

The reset link carried a long token ending in ==, which is a classic sign of Base64 encoding. So the obvious move was: don't treat it as a black box. Decode it and see what's inside. Using my own token, which I had already used, it decoded to something like this:

```
token (decoded):  
  <32 random hex chars> /// <32 random hex chars>  
  
e.g.  a1b2...c3d4 /// e5f6...a7b8  (values redacted)
```

This is where I expected to find something juicy. Instead I found the opposite: two long, random values with no email, no user ID, no timestamp, nothing personal at all. That's actually a textbook **secure** design. The randomness is huge, so guessing it is off the table, and there's no hidden data to abuse. The flow was also protected by a CAPTCHA and the link expired after 15 minutes.

I still tested the last question. I reset my password, then tried the same link again. The server rejected it with an "invalid request." It could only be used once, exactly as it should be.

The lesson I didn't expect

I came in hoping to find a flaw. I left having *confirmed there wasn't one*, and that flipped how I think about this whole thing. Security research isn't only about finding holes. It's about replacing assumptions with evidence. "This looks secure" is a guess. "I decoded the token, it's random, it doesn't leak data, and it can only be used once" is a verified answer.

Proving something is built well is a real result. It trains your eye to tell the difference between a system that's actually solid and one that just looks fine on the surface. That's the eye you need before you'll ever spot the real thing.

If you're just starting out

- **Read the scope twice.** Test only what you're allowed to, and only your own accounts.
- **Get your tools clean first.** A proxy that isn't set up properly will waste hours before you test anything real.
- **Never treat a token as a black box.** Decode it. You'll learn a lot even when it's boring.
- **Verify, don't assume.** A confirmed "no bug" beats a guessed "probably fine."
- **Practice on labs built for learning too.** Free platforms like PortSwigger's Web Security Academy teach the patterns faster than a busy live program will.

Note: all testing was done on my own test account, strictly within an authorized program's scope on Bugcrowd. No other users' data was ever accessed, and no sensitive values are reproduced here. This write up documents a learning process, not a vulnerability.